

Project Write-Up

Abstract

This report documents the design and implementation of a role-based desktop application for Shallom High School. The system authenticates six categories of school user and presents each with a dashboard tailored to their responsibilities, built entirely with the Java Swing toolkit to keep the project dependency-free and straightforward to run in any standard Java IDE. The report covers the problem being solved, the object-oriented design used to solve it, the event-driven programming model Swing imposes, security considerations, testing performed, and directions for extending the system beyond its current in-memory scope.

1. Introduction

Shallom High School required a single application that different members of the school community — the Headmaster, the Deputy Headmaster, a Teacher in Charge (class teacher), a subject Teacher, Ancillary Staff, and Students — could each open and immediately find the information relevant to them, without navigating a shared, undifferentiated interface. This project answers that requirement with a desktop application rather than a web service, so that it can be demonstrated and marked without any server setup, network configuration, or external dependency resolution.

2. Problem statement and objectives

- Authenticate a user against a fixed set of accounts and identify which of six roles they hold.
- Present only the navigation options and information relevant to that role.
- Communicate the school's identity: its motto, the subjects it teaches, and its physical facilities.
- Give administrative roles visibility of a master timetable and of which teacher heads which class.
- Explain, from the student's own dashboard, how school fees can be paid.
- Keep the solution runnable with nothing beyond a standard JDK, to remove build-tool friction from both development and marking.

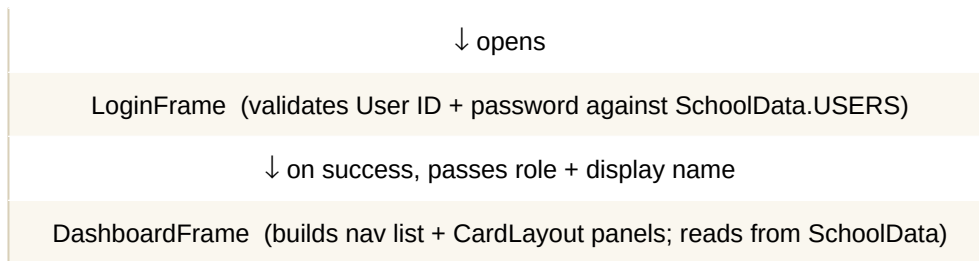
3. System architecture

The application follows a lightweight Model-View pattern adapted to Swing's event-driven style, spread across four classes:

Class	Role in the architecture
SchoolData	The Model. Holds every piece of school data as public static fields: the account table (User ID → password), the subject table (Subject ID → subject name), the class table (Class ID → class name), the teacher table (Teacher ID → teacher name), the timetable table (Class ID → Teacher ID → Timetable ID → Timetable name), and the fee table (Student ID → Fee ID → Fee amount).
LoginFrame	A View/Controller hybrid. Renders the login form, reads what the user typed, checks it against SchoolData, and either shows the DashboardFrame or shows an error message.
DashboardFrame	The main View after authentication. Builds a role-specific navigation list from a lookup table (navFor), and shows the selected dashboard.
Main	Composition root. Its only responsibility is to start the Swing event thread and show the first LoginFrame.

Figure 1: Control flow between the four classes.





4. Object-oriented and event-driven design decisions

- **Single source of truth:** every panel in DashboardFrame reads from SchoolData rather than holding its own copies of subjects, facilities, or timetable data, so a change in one place is reflected everywhere it is displayed.
- **Data-driven navigation:** the sidebar for each role is generated from a small `LinkedHashMap<String,String>` (navFor) rather than six hand-built layouts, so adding or reordering a role's sections is a one-line change rather than a new class.
- **CardLayout over multiple windows:** switching sections swaps a card in a single DashboardFrame instead of opening a new JFrame per section, which keeps window management simple and avoids orphaned windows if the user navigates quickly.
- **Event-driven validation:** LoginFrame reacts to a button click and to the Enter key on the password field via the same ActionListener, so keyboard and mouse users get identical behaviour.

5. Roles and responsibilities implemented

Role	What they see
Headmaster	School-wide overview, master timetable, class-teacher list, subjects, facilities, fee methods
Deputy Headmaster	Master timetable, class-teacher list, discipline log, facilities
Teacher in Charge	Their own class overview, class timetable, class register, subjects
Teacher	Personal overview, weekly timetable, list of classes taught, subjects
Ancillary Staff	Assigned duty, weekly duty roster, facilities
Student	Class and class-teacher summary, personal timetable, subjects, fee balance and payment methods

6. Security considerations

Because this is a standalone desktop demonstration rather than a networked service, authentication is intentionally simple: LoginFrame compares a typed User ID and password against plaintext values held in SchoolData.USERS. This is appropriate for coursework, where the goal is to demonstrate role separation, but it is explicitly not production-grade — the report's recommended extension path (Section 8) is to move authentication to a database with hashed passwords once the project grows beyond a single-machine demonstration.

7. Testing performed

- Compiled all four source files with javac to confirm the project builds cleanly with no external dependencies before distribution.

- Logged in with each of the six demo accounts in turn and confirmed the navigation list and every panel matched the role (e.g. only the Student account shows the Fees section with a balance).
- Tested invalid credentials (wrong password, unrecognised User ID) to confirm LoginFrame shows an error message rather than proceeding.
- Tested the Enter key on the password field as an alternative to clicking Log In.
- Tested Log Out from the dashboard to confirm it returns cleanly to a fresh LoginFrame.

8. Limitations and future work

- **No persistence:** all data resets when the application closes; a future version should read and write to a real database (e.g. via JDBC or JPA) so fee payments and register entries could actually be recorded.
- **Single sample timetable:** only Form 4A's timetable is modelled; a fuller version would store a distinct timetable per class and let an administrator edit it from within the application.
- **Plaintext credentials:** acceptable for a coursework demonstration, but should be replaced with hashed passwords and a proper user store before any real deployment.
- **No automated tests:** testing so far has been manual; adding JUnit tests around SchoolData and the login-validation logic would make regressions easier to catch as the project grows.

9. Conclusion

The finished application satisfies the brief set out in Section 2: a single entry point authenticates six distinct roles and routes each to a dashboard containing only the sections relevant to their responsibilities, while also surfacing the school's identity, staffing, timetable and fee information in one place. Building it purely in Swing, without a database or build tool, kept the project easy to compile and demonstrate, while the architecture — a single data model shared by a login view and a role-driven dashboard view — leaves a clear path toward a persistent, multi-class version should the project be extended.